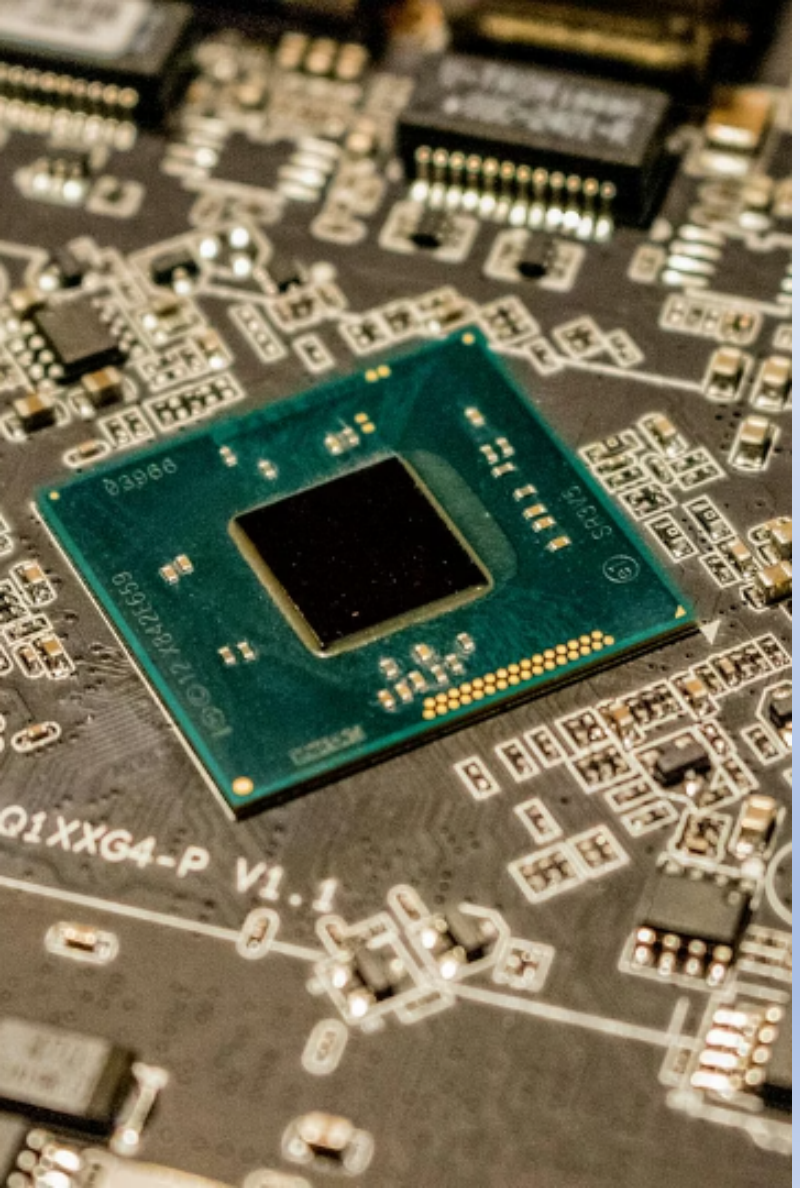


Merge Sort Implementation and Analysisusing RISC-V Architecture

Computer Organization and Architecture (COA) Project

Group 14

Nadheem Mujeeb · Joel Jogi · Aswin Raj R · Firdhouse Shinaj

Faculty: Athira Miss

What is Merge Sort?

Divide and Conquer

Recursively splits array into halves until single elements remain, then merges them back in sorted order.

Key Advantages

- Stable sorting algorithm
- Predictable O(n log n) time
- Efficient for large datasets
- Works well with linked lists

O(n log n)

Time Complexity

Best, average, and worst case

O(n)

Space Complexity

Requires temporary array

How Merge Sort Works

1

Divide

Split array into two halves recursively until elements are singletons

2

Sort

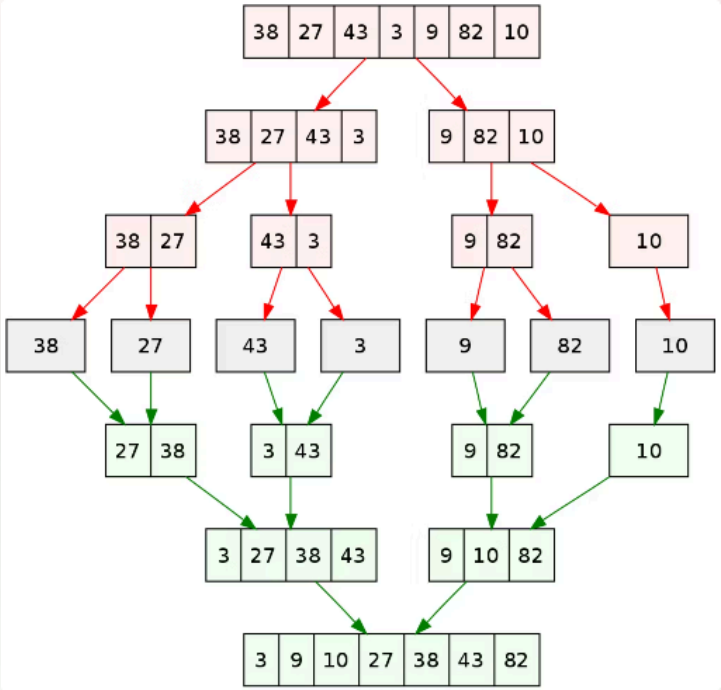
Base case reached when single elements exist (already sorted)

3

Merge

Combine pairs while maintaining ascending order

Example: Array [38, 27, 43, 3, 9, 82, 10] splits into [38, 27, 43, 3] and [9, 82, 10], continues dividing, then merges as [3, 27, 38, 43] and [9, 10, 82], finally combining into [3, 9, 10, 27, 38, 43, 82]



C Code Implementation

merge() Function

```c
void merge(int arr[], int left, int mid, int right) {
    int i = left;
    int j = mid + 1;
    int k = left;

    int temp[100];  // temporary array

    // Merge two halves
    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp[k] = arr[i];
            i++;
        } else {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }

    // Copy remaining elements
    while (i <= mid) {
        temp[k] = arr[i];
        i++; k++;
    }

    while (j <= right) {
        temp[k] = arr[j];
        j++; k++;
    }

    // Copy back to original array
    for (i = left; i <= right; i++) {
        arr[i] = temp[i];
    }
}
```

mergeSort() + main()

```c
void mergeSort(int arr[], int left, int right) {
    if (left >= right)
        return;

    int mid = (left + right) / 2;

    mergeSort(arr, left, mid);        // left half
    mergeSort(arr, mid + 1, right);   // right half
    merge(arr, left, mid, right);     // merge
}

int main() {
    int arr[] = {5, 2, 9, 1, 6};
    int n = 5;

    mergeSort(arr, 0, n - 1);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

RISC-V Assembly

Memory Access

`lw` (load word) and `sw` (store word) transfer data between memory and registers

Arithmetic

`add`, `sub`, `addi` perform calculations and address computations

Control Flow

`jal` (jump and link) and `ret` (return) manage recursive function calls

Branching

`blt` (branch if less than) implements loop and conditional logic

RISC-V Part 1: Data & Main Function

```
# Data section
.data
array: .byte 5, 2, 9, 1, 6
temp:  .byte 5

# Text section
.text
.globl main

# Main entry point
main:
    la a0, array      # load array address
    addi a1, a0, 4    # last index
    jal ra, mergesort # call mergesort
    j print_result    # print results
```

Data Section:

- `array`: Stores the unsorted array [5, 2, 9, 1, 6]
- `temp`: Temporary array for merge operations

Main Function:

- `la a0, array`: Load the base address of the array into register a0
- `addi a1, a0, 4`: Calculate the last index (array address + 4 bytes)
- `jal ra, mergesort`: Jump to mergesort function, save return address in ra
- `j print_result`: After sorting, jump to print the results

Key Registers:

- `a0`: First argument (array address)
- `a1`: Second argument (right index)
- `ra`: Return address

RISC-V Part 2: mergeSort Function

```
# Recursive merge sort
mergesort:
    addi sp, sp, -16  # allocate stack
    sw ra, 0(sp)      # save return addr
    sw a0, 4(sp)      # save left
    sw a1, 8(sp)      # save right

    bge a0, a1, ms_end # if left >= right

    # Calculate mid
    add t0, a0, a1
    srli t0, t0, 1    # mid = (l+r)/2

    # Sort left half
    mv a1, t0
    jal ra, mergesort

    # Restore and sort right half
    lw a0, 4(sp)
    lw a1, 8(sp)
    add t0, a0, a1
    srli t0, t0, 1

    addi t1, t0, 1
    mv a0, t1
    lw a1, 8(sp)
    jal ra, mergesort

    # Merge sorted halves
    lw a0, 4(sp)
    lw a1, 8(sp)
    add t0, a0, a1
    srli t0, t0, 1

    mv a2, a1
    mv a1, t0
    jal ra, merge

ms_end:
    lw ra, 0(sp)      # restore return
    addi sp, sp, 16   # deallocate
    ret
```

Stack Management:

- Allocates 16 bytes on stack to save return address and parameters
- Saves ra, left (a0), and right (a1) for recursive calls

Divide Phase:

- Calculates mid = (left + right) / 2 using add and srli (shift right logical immediate)
- Recursively calls mergeSort on left half [left, mid]

Conquer Phase:

- Restores saved values from stack
- Recursively calls mergeSort on right half [mid+1, right]

Combine Phase:

- Calls merge() to combine the two sorted halves
- Merge function receives: a0=left, a1=mid, a2=right

Base Case:

- When left >= right, the array is already sorted (single element)
- Restores return address and deallocates stack space
- Returns to caller with ret instruction

RISC-V Part 3: merge() & print_result()

```
#merge two sorted halves
merge:
    la t6, temp       # temp array
    mv t0, a0         # i = left

    addi t1, a1, 1    # j = mid+1

merge_loop:
    bgt t0, a1, copy_right

    bgt t1, a2, copy_left

    lb t3, 0(t0)      # load arr[i]


    lb t4, 0(t1)      # load arr[j]

    ble t3, t4, take_left

    sb t4, 0(t6)      # store arr[j]

    addi t1, t1, 1

    j next

take_left:

    sb t3, 0(t6)      # store arr[i]

    addi t0, t0, 1
next:
    addi t6, t6, 1

    j merge_loop
copy_left:

    bgt t0, a1, copy_done

    lb t3, 0(t0)

    sb t3, 0(t6)
    addi t0, t0, 1
    addi t6, t6, 1
    j copy_left
copy_right:
    bgt t1, a2, copy_done
    lb t4, 0(t1)
    sb t4, 0(t6)
    addi t1, t1, 1
    addi t6, t6, 1
    j copy_right
copy_done:
    la t6, temp
```

```
copy_back:

    bgt t0, a2, merge_end

    lb t3, 0(t6)    sb t3, 0(t0)
    addi t6, t6, 1
    addi t0, t0, 1
    j copy_back
merge_end:
    ret
# Print sorted array
print_result:
    la t0, array      # array address
    li t1, 5          # array size
print_loop:
    lb a0, 0(t0)      # load element
    li a7, 1          # print int
    ecall
    li a0, 32         # space char
    li a7, 11         # print char
    ecall
    addi t0, t0, 1
    addi t1, t1, -1
    bnez t1, print_loop
    li a7, 10         # exit
    ecall
```

merge() Function:

- Compares elements from left and right halves
- Stores smaller element in temp array
- Handles remaining elements when one half is exhausted
- Copies merged result back to original array

print_result() Function:

- Loads array address and size
- Iterates through each element
- Uses ecall to print each number and space
- Exits program with ecall (a7=10)

Single Cycle Datapath

01

Instruction Fetch (IF)

Retrieve instruction from memory

02

Instruction Decode (ID)

Decode opcode and read registers

03

Execute (EX)

Perform ALU operation

04

Memory Access (MEM)

Load/store data from memory

05

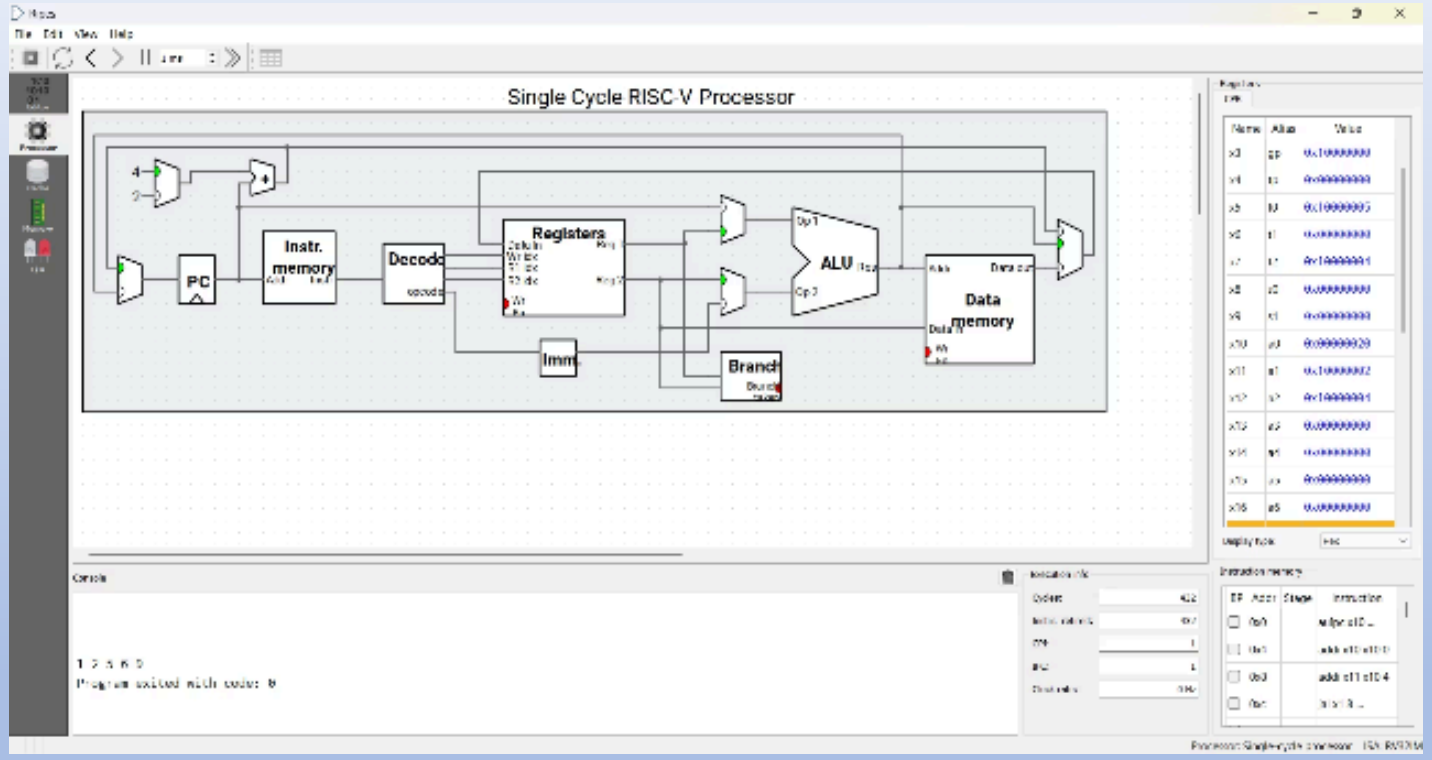
Write Back (WB)

Update register with result

Characteristics

- One instruction per clock cycle
- Simple hardware design
- Slow performance
- High CPI (cycles per instruction)

Single Cycle Datapath



Data flows from left to right as an instruction is fetched, decoded, executed, and then written back in one clock cycle. Instruction memory supplies the instruction, the register file provides operands, the ALU performs the calculation, and data memory is used for loads and stores. The final result then travels back through the write-back path to update the register file.

Single Cycle Datapath: Execution Analysis with RISC-V

```
# Example: Load and Add Instructions
lw x5, 0(x1)      # Load word from memory
addi x6, x5, 10   # Add immediate
sw x6, 4(x1)      # Store word to memory
```

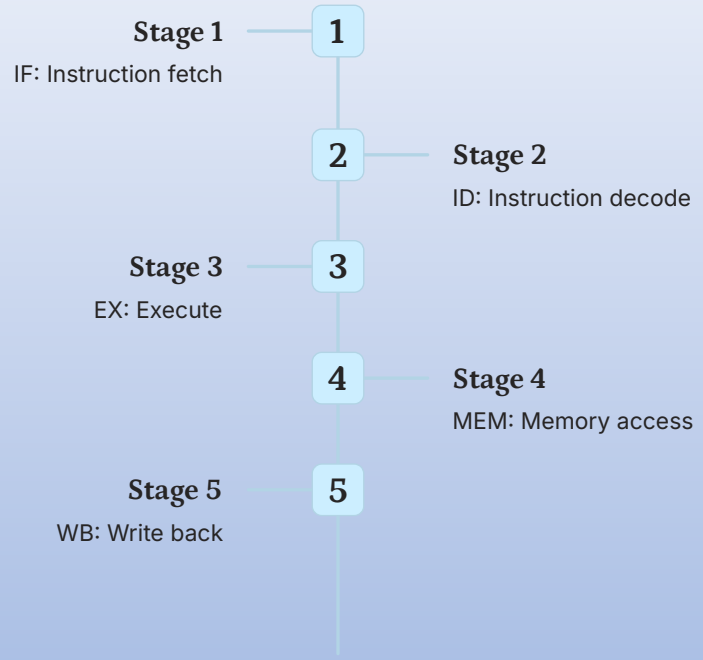
Execution Flow

- **Instruction Fetch (IF)**: PC → Memory → Instruction Register
- **Instruction Decode (ID)**: Parse opcode, read source registers (x5, x1)
- **Execute (EX)**: ALU performs addition (x5 + 10)
- **Memory Access (MEM)**: Write result to memory at address (x1 + 4)
- **Write Back (WB)**: Update destination register x6

Performance Metrics

- **Each instruction completes in 1 clock cycle (CPI = 1)**
- **Clock cycle is long because all stages execute together in one cycle**
- **Total Execution Time**: 1 × Clock Period
- **Throughput**: 1 instruction every clock cycle
- **Hardware Complexity**: Low (simple control logic)
- **Bottleneck**: Long critical path limits clock frequency

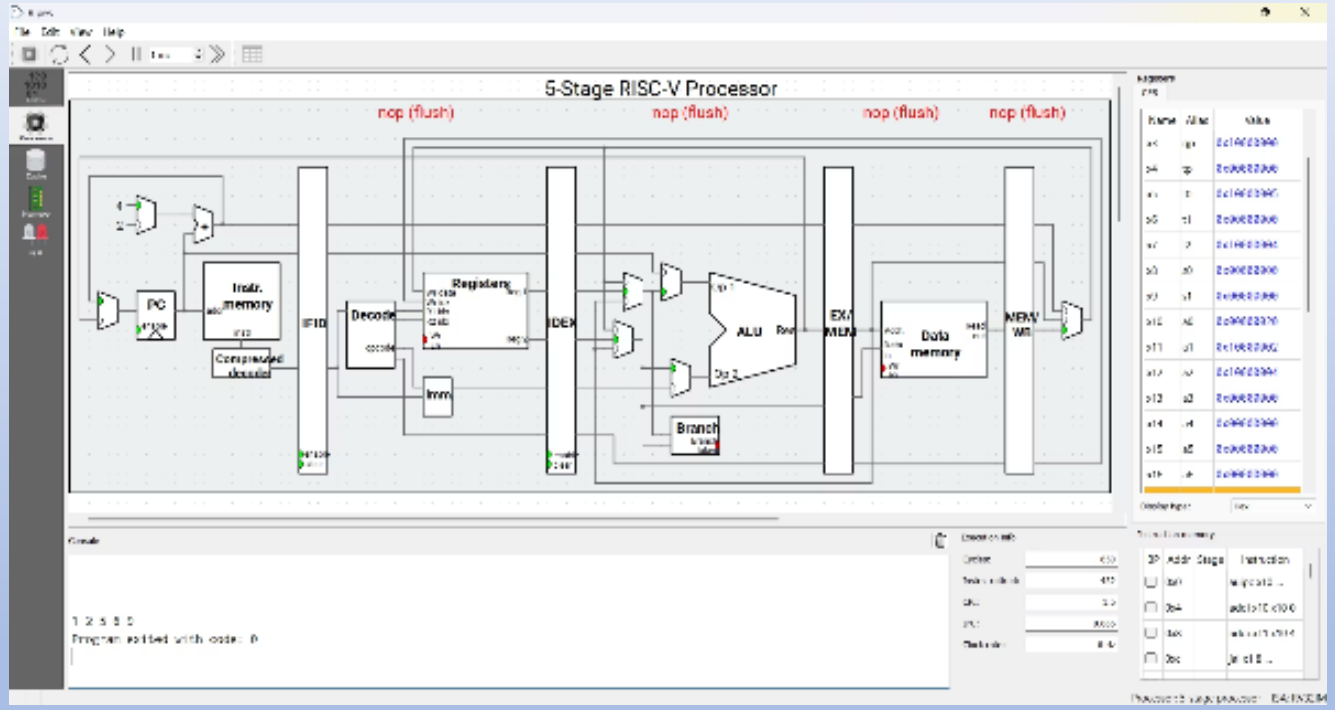
Pipelined Datapath



Key Advantages

- Multiple instructions in flight simultaneously
- Better throughput
- Lower effective CPI
- Improves throughput but may not always reduce total execution time

Pipelined Datapath



An instruction flows from left to right through the pipeline stages: fetch, decode, execute, memory access, and write back. The vertical rectangular blocks in the diagram represent pipeline registers that hold instruction state between stages. Each stage performs one part of the work, while the vertical blocks between them act as pipeline registers that hold and pass data to the next stage. This lets multiple instructions move through different stages at the same time, improving overall throughput.

Pipelined Datapath: Execution Analysis with RISC-V

Example: Load and Add Instructions (5-stage pipeline)

```
# Example: Load and Add Instructions (5-stage pipeline)
lw x5, 0(x1)      # Load word from memory
addi x6, x5, 10   # Add immediate
sw x6, 4(x1)      # Store word to memory
```

Note: The theoretical lw-addi-sw example above is simplified for illustration. Real merge sort execution is more complex.

- **Instructions retired:** 432
- **Single cycle:** 432 cycles
- **Pipelined:** ~650 cycles

Pipeline execution took more cycles due to hazards and the recursive nature of merge sort, which introduces data dependencies and stalls.

Pipeline Hazards

Data Hazard

When an instruction depends on data from a previous instruction not yet written back. Solved via forwarding or stalls.
Example: `add x5, x1, x2` followed by `sub x6, x5, x3` requires x5 value

Control Hazard

Branch instructions cause uncertainty about which instruction to fetch next. Handled via branch prediction or flushing.
Example: `beq x1, x2, label` requires knowing branch target

Structural Hazard

Multiple instructions need same hardware resource simultaneously. Resolved by duplicating resources or stalling.
Example: Two instructions both need memory access in same cycle

Performance Comparison & Speedup - Ripes Simulation Results

Metric	Single Cycle	5-Stage Pipeline
Instructions	432	432
Total Cycles	432	650
CPI	1.0	~1.5
Pipeline Stages	1	5 (IF, ID, EX, MEM, WB)
Hazard Handling	None	Forwarding, stalls

What We Observed:

- Both execute the same 432 instructions
- Single cycle: 1 instruction per cycle (no overhead)
- Pipeline: Takes more cycles due to filling and hazards
- But pipeline can execute multiple instructions simultaneously

Why Pipeline Has More Cycles:

- Pipeline filling (stages need to fill at start)
- Data hazards causing stalls
- Branch mispredictions
- Memory access delays

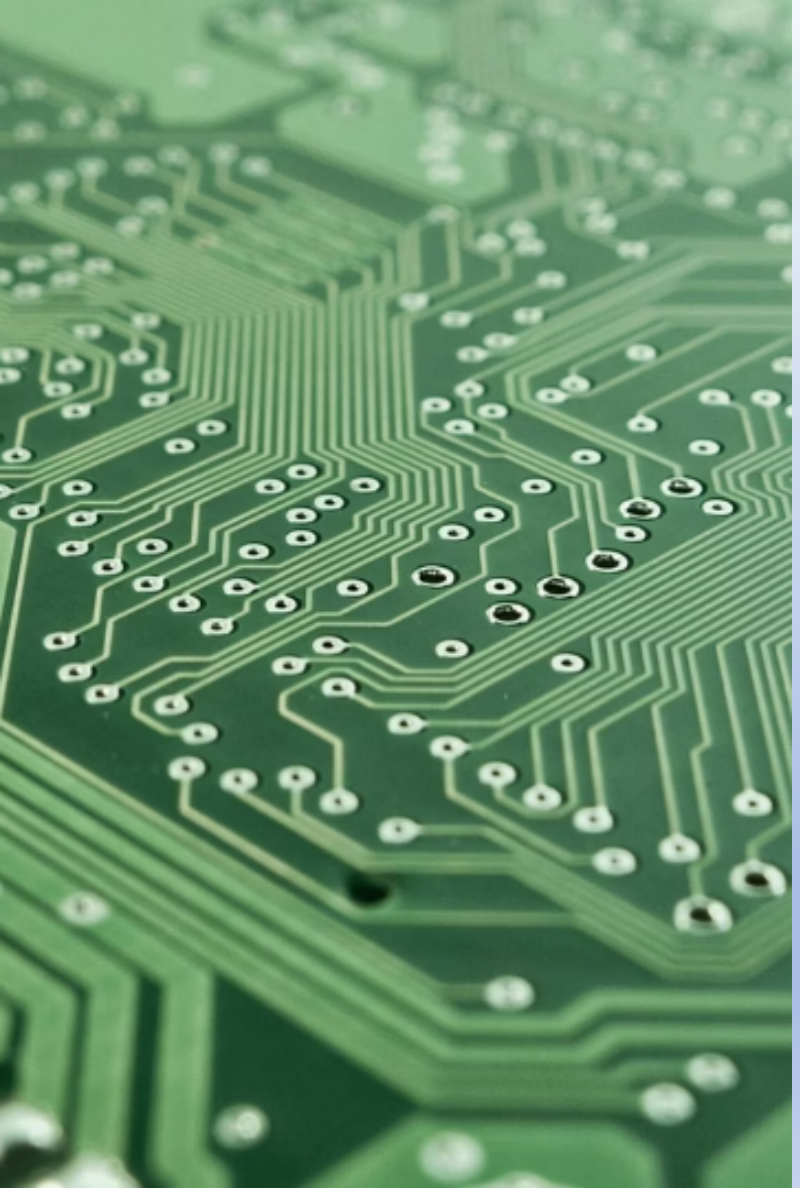
Performance Trade-off:

- Single cycle: Simple hardware, but slower throughput
- Pipeline: Complex hardware, but better instruction throughput
- Theoretical speedup: ~5× (due to 5 stages)
- Practical speedup: Depends on code and hazards

Key Point:

- Pipelining trades complexity for better performance
- Modern CPUs use pipelining because throughput matters

Results from Ripes simulator



Control Signals & Conclusion

Control Unit Role

Generates control signals that determine ALU operation, memory read/write, register enable, and data path routing for each instruction.

Key Control Signals

- ALU control (add, subtract, etc.)
- Memory read/write enable
- Register write enable
- Branch control
- Jump control

Final Conclusion

Merge Sort is Efficient

O(n log n) time complexity makes it suitable for large datasets and predictable performance

Pipeline Architecture Improves Performance

5-stage pipelining achieves ~5× speedup over single cycle implementation by executing multiple instructions simultaneously. Recursive calls and data dependencies in merge sort introduce stalls in the pipeline, which is why the Ripes simulation shows ~650 cycles instead of the theoretical speedup.

Real-World Trade-offs

Hardware complexity increases with pipeline stages and hazard handling mechanisms